



COMPOSITION NOTEBOOK

Models of Computation

Syntax of lambda

The calculus of lambda (λ -calculus) is an essential mathematical language that lies at the foundation of programming languages.

- Var (set of variables): its the finite set of the variables, i.e.: $x, y, z, \dots$;
- Λ (set of lambda-terms): its the set of all lambda terms, that is all valid expression that we can build.

We can define Λ with this formula:

$$\Lambda ::= \text{Var} \mid \Lambda \Lambda \mid (\lambda \text{Var}. \Lambda)$$

if CAN BE COMPOSED OF

So:

- Var:

$$\frac{m \in \text{Var}}{m \in \Lambda}$$

if m is a variable ($m \in \text{Var}$), THEN m is a valid term ($m \in \Lambda$)

- Application:

$$\frac{M \in \Lambda \quad N \in \Lambda}{(M N) \in \Lambda}$$

if M is a valid term ($M \in \Lambda$) and N too ($N \in \Lambda$), then we can write them next to each other ($M N \in \Lambda$)

- Abstraction:

$$\frac{m \in \text{Var} \quad M \in \Lambda}{(\lambda m. M) \in \Lambda}$$

if a variable $m \in \text{Var}$ and M is a valid term, then we can write $\lambda m. M$ and we obtain a new valid term

Exercise 1:

$$f(a, b) = a + b$$

We change the terms to make them more recognizable.

$$f(m, y) = m + y$$

We replace m with 3 and y with 7.

$$f(3, 7) = 3 + 7 = 10$$

λ can accept only one variable at a time. We can't write $\lambda(m, y)$, so we apply Abstraction (ABS)

2 times, one for m and one for y .

$$\frac{y \in \text{Var} \quad (m + y) \in \Lambda}{(\lambda y. m + y) \in \Lambda}$$

Subsequently, we apply it to m :

$$\frac{m \in \text{Var} \quad (\lambda y. m + y) \in \Lambda}{\lambda m (\lambda y. m + y) \in \Lambda}$$

So: $\lambda m (\lambda y. m + y)$

$$f = \lambda m. (\lambda y. m+y) \Rightarrow (\lambda m y. m+y)$$

Now, if we pass a number to the function, e.g. 3

$$(\lambda m y. m+y) 3$$

Subsequently, λm comes off because its mission is concluded

$$\lambda y. 3+y$$

We can stop here because with the next number i.e. 7 in this case, the sum will be equal to 10!

Exercise 2:

REMEMBER! THE PARENTHESES ARE HIDDEN. THE TRUE FORM IS: $\lambda m (\lambda y (\lambda z. m z (y z)))$

$$S = \lambda m y z. m z (y z)$$

Looking all the way to the left, we would have λm , so this is the first we delete

$$\boxed{\lambda m} . \boxed{(\lambda y z. m z (y z))}$$

m y z

using the abs rule $\frac{m \in \text{Var} \quad H \in \Lambda}{(\lambda m. H) \in \Lambda}$ bottom up, so:

... $\in \text{Var} / \Lambda$ WE CAN ALSO NOT WRITE IT EVERYWHERE

$$m \in \text{Var} \quad \lambda y z. m z (y z) \in \Lambda$$

Now we apply abs to the y :

$$y \in \text{Var} \quad \lambda z. m z (y z)$$

and now the same for z :

$$z \in \text{Var} \quad m z (y z)$$

Now we apply the Application rule (app) to the rule $m z$ and $y z$:

$$\boxed{m z} \quad \boxed{(y z)}$$

m z y z

$m z \in \Lambda$ $y z \in \Lambda$

We re-apply app to $m z$ and $y z$:

$$\boxed{m} \quad \boxed{z} \quad \boxed{y} \quad \boxed{z}$$

$m \in \Lambda$ $z \in \Lambda$ $y \in \Lambda$ $z \in \Lambda$

In the end we apply the Var rule to all 4 variables:

$$\boxed{m} \quad \boxed{z} \quad \boxed{y} \quad \boxed{z}$$

$m \in \text{Var}$ $z \in \text{Var}$ $y \in \text{Var}$ $z \in \text{Var}$

Calculus of Lambda: α -rule, β -reductions

α -RULE: the name we give to an "internal" variable in the function doesn't change the sense of the same function's meaning

β -REDUCTION: it's a lambda calculus rule and it applies a function to the value (substitution)

$$(\lambda m. H) N \xrightarrow{\beta\text{-REDUCTION}} H[N/m] \rightarrow \text{REDUCT}$$

Exercise 3:

$$S = (\lambda yz. m z (y z)) (\lambda a b. a) (\lambda c d. c)$$

STEP 1: β -reduction

$(\lambda a b. a)$ is an input value, being the first variable m disappears and is replaced:

$$(\lambda y z. (\lambda a b. a) z (y z)) (\lambda c d. c)$$

STEP 2: 2° β -reduction

we do the same with y and $(\lambda c d. c)$

$$\lambda z. (\lambda a b. a) z ((\lambda c d. c) z)$$

STEP 3.1: β -reduction

we apply the rule to the a
with z in input

$$\lambda b. z$$

STEP 3.2: β -reduction

we apply the rule to the c
with z in input

$$\lambda d. z$$

So:

$$\lambda z. (\lambda b. z) (\lambda d. z)$$

STEP 4: β -reduction

now we apply the rule to the b with $(\lambda d. z)$ in input

$$\lambda z. z$$

BEING THAT b DOESN'T EXIST TO THE RIGHT OF THE DOT, I CAN'T REPLACE ANYTHING

In this case, when we have a substitution like: $\lambda m. m$, $\lambda a. a$ etc..., we write I (Identity):

$$\lambda z. z = I$$

Contextual β -reduction

Now let's look at the compatibility rules.

• REGOLA (μ):

$$\frac{M \xrightarrow{\beta} M'}{M N \xrightarrow{\beta} M' N}$$

if the part M changes into M' , then inside the pair MN you can change the left part to get $M'N$ (leaving N unchanged)

• REGOLA (η):

$$\frac{N \xrightarrow{\beta} N'}{HN \xrightarrow{\beta} MN'}$$

if the part N changes into N' , then inside the pair HN you can change the right part to get MN' (leaving H unchanged)

• REGOLA (ξ):

$$\frac{M \xrightarrow{\beta} M'}{\lambda m. M \rightarrow \lambda m. M'}$$

if the body changes into M' , then you can do that step even if there is $\lambda m.$ in front, getting $\lambda m. M'$

$$M \twoheadrightarrow M': \text{SEVERAL STEPS}$$

The arrow ($\rightarrow$) means "I do only one calculation step".

The two-headed arrow ($\twoheadrightarrow$) means "I do many calculation steps in a row" (or even zero steps) to go from M to M' .

Church-Rosser Theorem

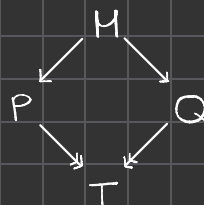
Imagine we have the expression M , for example $(7+4)(8+5 \cdot 3)$

We can choose to calculate the left part first, getting the intermediate result P .

Or we can choose to calculate the right part first, getting the intermediate result Q .

In any case, after several steps we still arrive at T .

So the order of the calculations does not change the final result!



The infinite loop

$$(\lambda x. xx)(\lambda x. xx)$$

So I have $(\lambda x. xx)$ and the input $(\lambda x. xx)$, substituting the first x we get $(\lambda x. xx)x$, and then the second x :

$$(\lambda x. xx)(\lambda x. xx)$$

So it is a real loop

Notable Shortcut

From now on, letters will be used to replace the most common functions:

- $I \equiv \lambda m. m$ (Identity)
- $K \equiv \lambda m y. m$ (Constant)
- $S \equiv \lambda m y z. m z (y z)$ (Substitution)
- $B \equiv \lambda m y z. m (y z)$ (Composition)

- $C \equiv \lambda w y z. w z y$ (Swap)

Exercise 4:

$S(BBS)(KK)$

$(\lambda m y z. m z (y z))(BBS)(KK)$

$(\lambda z. BBS z (KKZ)) \rightarrow (\lambda m y. m) K Z \rightarrow (\lambda y. K) z \rightarrow K$

$\lambda z. BBS z K$

$\lambda z. (\lambda m y z. m (y z)) B S z K$

$\lambda z. (\lambda y z. B (y z)) S z K$

$\lambda z. (\lambda z. B (S z)) z K$

$\lambda z. B (S z) K$

⚠ In this case $B \equiv \lambda a b c. a (b c)$, otherwise the z's will get mixed up

$\lambda z (\lambda a b c. a (b c)) (S z) K$

$\lambda z (\lambda b c. S z (b c)) K$

$\lambda z (\lambda c. S z (K c))$

$\lambda z c. S z (K c)$

→ SAME THING: I ALREADY HAVE Z, SO IN THIS CASE $S \equiv \lambda u v t. u t (v t)$

$\lambda z c. (\lambda u v t. u t (v t)) z (K c)$

$\lambda z c. (\lambda v t. z t (v t)) (K c)$

$\lambda z c. (\lambda t. z t (K c t))$

$\lambda z c t. z t (\lambda m y. m) c t$

$\lambda z c t. z t (\lambda y. c)$

$\lambda z c t. z t c \equiv C$

Boolean

In Lambda Calculus, truth values are simulated using functions that chose between two options:

• True:

$TRUE \equiv \lambda m y. m$

It's identical to the K combinator. So it always picks the first one (m) and ignores the second one,

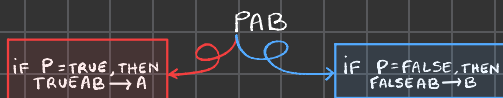
• False:

$FALSE \equiv \lambda m y. y$

it takes two arguments and always selects the second one (y), ignoring the first one

Example 5:

if P then A else B



Numerals

Numerals do not exist as digits or quantities, but they show how many times a function is repeated.

$m+1 \equiv \lambda m y. m(m(\dots(m y) \dots))$ ← THE FUNCTION m IS REPEATED $m+1$ TIMES ON THE BASE y

- $0 \equiv \lambda m y. y$: the action is repeated 0 times (it returns the base y unchanged doing nothing);
- $1 \equiv \lambda m y. m y$: the action is repeated 1 time;
- $2 \equiv \lambda m y. m(m y)$: the action is repeated 2 times in a row;
- $3 \equiv \lambda m y. m(m(m y))$: the action is repeated 3 times.

QUICK TRICK: JUST COUNT HOW MANY "m" THERE ARE. FOR EXAMPLE $0 \equiv \lambda m y. y$ BECAUSE AFTER THE DOT "," THERE ARE NO "m"s. $2 \equiv \lambda m y. m(m y)$ BECAUSE AFTER DOT "," THERE ARE 2 "m"s ETC...

Exercise 6

Find $\text{succ} \in \Lambda$ s.t. $\text{succ } m = m+1 \forall m$

← WE NEED TO CREATE A "succ" FUNCTION THAT TAKES ANY NUMBERS AS INPUT AND RETURNS THE NEXT ONE ($m+1$)

The formula to calculate it's:

$$\lambda x y. x(x y)$$

For example, let's take number 2, which is equal to: $\lambda m y. m(m y)$

a. 0 DISAPPEARS BECAUSE WE PASSED 2 AS INPUT

←

$$\lambda x y. x((\lambda m y. m(m y)) x y)$$

$$\lambda x y. x((\lambda y. x(x y)) y)$$

$$\lambda x y. x(x(x y))$$

← IF WE CHECK, IT WILL BE EQUAL TO 3

And what if we had $m = m+2$?

In that case, the formula would be: $\lambda x y. x(x(x y))$

We insert 1, which is equal to $\lambda m y. m y$:

$$\lambda x y. x(x((\lambda m y. m y) x y))$$

$$\lambda x y. x(x((\lambda y. x y) y))$$

$$\lambda x y. x(x(x y))$$

The test of zero

We want a function $\mathbb{Z}$ that tells us if a number is zero or not:

- $\mathbb{Z} 0 = \text{TRUE}$: if the number is 0, then it is true,
- $\mathbb{Z}(m+1) = \text{FALSE}$: if the number is 1 or higher, it is false.

Now let's see a proof of how it works, using the formula:

$$\mathcal{I} = \lambda a. a \left(\lambda x. \text{FALSE} \right) \text{TRUE}$$

$$\mathcal{I} = (\lambda a. a \text{ UV})$$

Let's see what happens if we insert $\mathcal{O}$ (so it should give TRUE, because that is what the formula is for):

$$(\lambda a. a \text{ UV}) \mathcal{O} = \mathcal{O} \text{ UV}$$

now looking at the definition of $\mathcal{O}$, which is $\lambda m. y$, therefore:

$$(\lambda m. y) \text{ UV} = y = \text{TRUE}$$

Church Numerals are iterators

A number is not a static quantity, but an "iterator", which means an engine that repeats an action a specific number of times.

Addition (Add)

$$\text{ADD } \mathcal{O} \text{ } n = n$$

→ 0 + n RETURNS n

$$\text{ADD } w + 1 \text{ } m = \text{succ}(\text{Add } w \text{ } m)$$

→ TO ADD H+1 TO N, WE JUST CALCULATE THE SUM OF H AND N AND THEN APPLY THE SUCCESSOR.

The official formula to add two numbers is:

$$\text{ADD} \equiv \lambda m. y. m \text{ succ } y$$

When we pass any two numbers (called w and m) to this formula, the calculation works like this:

$$\begin{aligned} \text{Add } w \text{ } m &= (\lambda m. y. m \text{ succ } y) w m \\ &= w \text{ succ } m \end{aligned}$$

How it works: for example, if we need to do $2+3$, the number 2 applies the successor 2 times starting from 3, getting 5.

We can generalize the mechanism we just explained:

- $F_0 = R_0$: this is the base case or starting point. It means that if the loop runs zero times, the result is simply the initial value (called R_0);
- $F_{(n+1)} = R_1(F_n)$: this is the step forward. It means that to take an extra step, you take the result from the previous loop (F_n) and apply a new action (called R_1) to it.

$$m \text{ } R_1 \text{ } R_0$$

So any Church numeral m always works the same way: it takes an action (R_1) and runs it m times in a row, starting from a base (R_0).